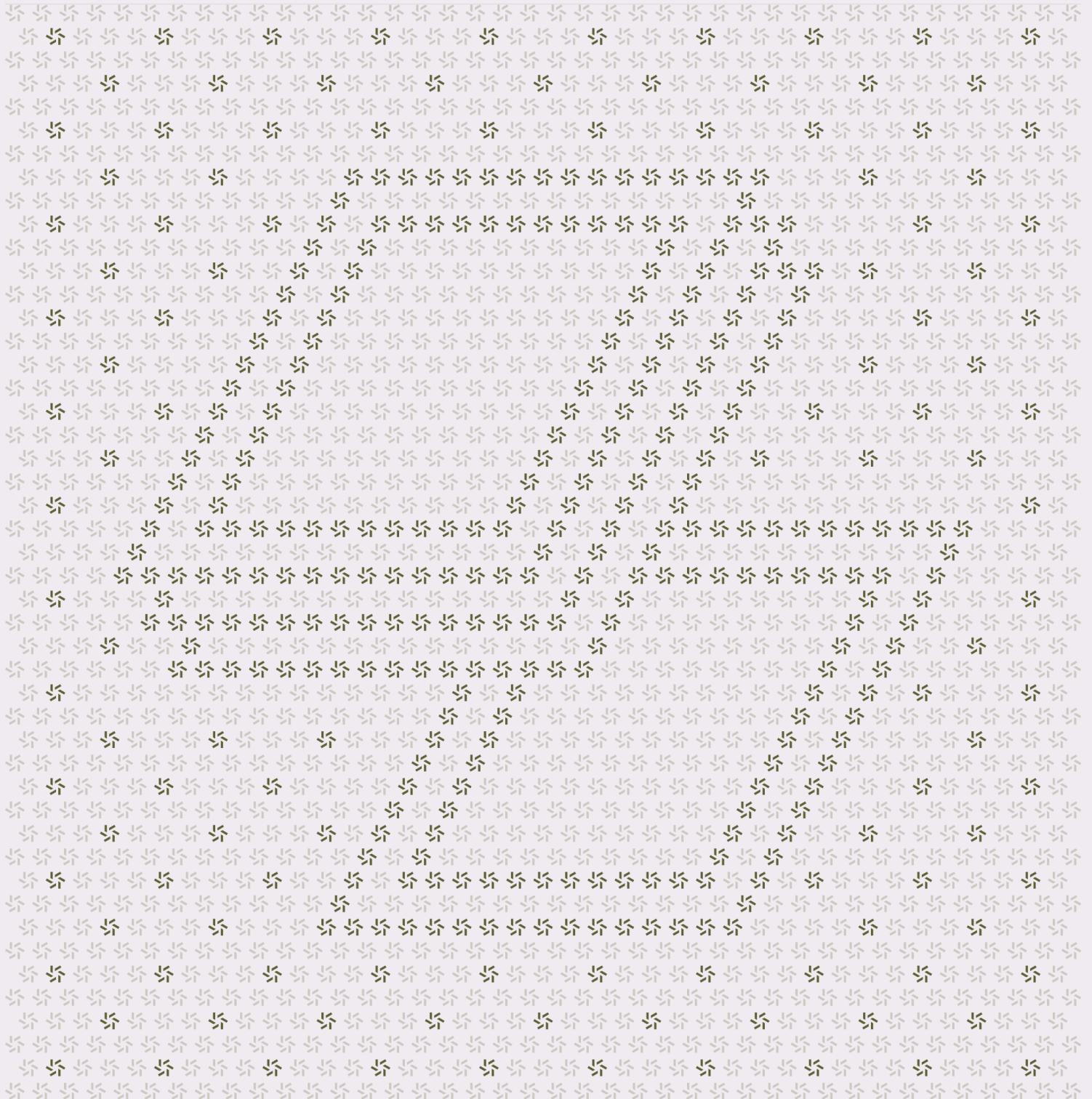


December 9, 2025

VM Rewrite

Program Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	6
2. Introduction	6
2.1. About VM Rewrite	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	13
2.5. Project Timeline	13
3. Detailed Findings	13
3.1. Native function memory leak in Arena-allocated Function structures	14
3.2. Unbounded package-cache growth can lead to memory exhaustion	16
3.3. Potential integer overflow in bytecode offset calculation	18
3.4. Assert-based panic in interpreter	20
3.5. Gas cost double-charged in native_sub_string	22
4. System Design	23
4.1. Overview	24

4.2.	Command type: Publish	25
4.3.	Command type: MoveCall	28
4.4.	Command type: TransferObjects	30
4.5.	Command type: SplitCoins	31
4.6.	Command type: MergeCoins	33
4.7.	Command type: MakeMoveVec	34
4.8.	Command type: Upgrade	36
4.9.	Attack surface	37

5.	Assessment Results	39
5.1.	Disclaimer	40

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Mysten Labs from October 16th to December 5th, 2025. During this engagement, Zellic reviewed VM Rewrite's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any out-of-memory vulnerabilities or bounds calculation errors in the memory-allocation and resource-limitation mechanisms?
 - Could invalid or dangling pointers arise from low-level memory operations?
 - Are there any Rust borrowing or mutability violations that could compromise memory safety?
 - Is the implementation of Move language semantics in the VM correct and strictly enforced?
 - Could incorrect mutations of mutable references lead to multiple live mutable references pointing to the same value?
 - Are the permissions and ownership models strictly adhered to when objects are copied, moved, or read/written?
 - Are the V-Tables' loading mechanisms in the adapter correct and type-safe?
 - Could array or collection accesses lead to runtime panics from index out-of-bounds errors?
 - Are all uses of `unwrap()` and `expect()` properly handled to prevent unexpected panics?
 - Could deep recursion or circular dependencies lead to stack overflow?
 - Are synchronization primitives for shared state (such as `Mutex` or `RefCell`) used correctly to prevent deadlocks and race conditions?
 - Could JIT code generation introduce semantic errors or resource leaks?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Optimization passes
- Front-end components
- Infrastructure relating to the project

- Key custody

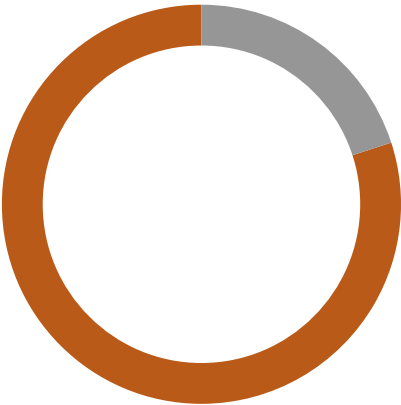
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped VM Rewrite targets, we discovered five findings. No critical issues were found. Four findings were of high impact and the remaining finding was informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	4
Medium	0
Low	0
Informational	1



2. Introduction

2.1. About VM Rewrite

Mysten Labs contributed the following description of VM Rewrite:

The Bella Ciao VM rewrite aims to modernize and streamline the Move virtual machine by removing many of the cumbersome design patterns that have accumulated over time. The new VM architecture aligns more closely with the Sui execution model and elevates packages to a first-class concept, replacing the legacy reliance on modules and "module bundles" as the core building block in the VM.

This rewrite is designed not only to simplify the VM's structure but also to enable deeper and more effective optimizations over time. It introduces a more advanced execution model, as well as internal bytecode representations, and reduces reliance on high-level Rust features, instead leveraging lower-level Rust mechanisms such as arenas for memory management to improve performance and control.

In addition to these structural improvements, the new design establishes a stronger foundation for future language features such as virtual dispatch (e.g., traits and related constructs).

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the targets.

Architecture risks. This encompasses potential hazards originating from the blueprint of a system, which involves its core validation mechanism and other architecturally significant constituents influencing the system's fundamental security attributes, presumptions, trust mode, and design.

Arithmetic issues. This includes but is not limited to integer overflows and underflows, floating-point associativity issues, loss of precision, and unfavorable integer rounding.

Implementation risks. This encompasses risks linked to translating a system's specification into practical code. Constructing a custom system involves developing intricate on-chain and off-chain elements while accommodating the idiosyncrasies and challenges presented

by distinct programming languages, frameworks, and execution environments.

Availability. Denial-of-service attacks are another leading issue in custom systems. Issues including but not limited to unhandled panics, unbounded computations, and incorrect error handling can potentially lead to consensus failures.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

VM Rewrite Targets

Type	Rust
Platform	Sui

Target	move-vm-runtime
Repository	https://github.com/MystenLabs/sui
Version	eed1f9d396e67d9e70e0311fa36bd84c939c583
Programs	cache/arena.rs cache/identifier_interner.rs cache/mod.rs cache/move_cache.rs execution/dispatch_tables.rs execution/interpreter/eval.rs execution/interpreter/helpers.rs execution/interpreter/locals.rs execution/interpreter/mod.rs execution/interpreter/state.rs execution/mod.rs execution/values/mod.rs execution/values/values_impl.rs execution/vm.rs jit/execution/ast.rs jit/execution/mod.rs jit/execution/translate.rs jit/mod.rs lib.rs natives/extensions.rs natives/functions.rs natives/mod.rs natives/move_stdlib/bcs.rs natives/move_stdlib/debug.rs natives/move_stdlib/hash.rs natives/move_stdlib/mod.rs natives/move_stdlib/signer.rs natives/move_stdlib/string.rs natives/move_stdlib/type_name.rs natives/move_stdlib/unit_test.rs natives/move_stdlib/vector.rs runtime/data_cache.rs runtime/mod.rs runtime/package_resolution.rs shared/binary_cache.rs shared/constants.rs shared/data_store.rs shared/gas.rs shared/linkage_context.rs shared/logging.rs shared/mod.rs shared/serialization.rs

Target	move-vm-runtime
Version	eed1f9d396e67d9e70e0311fa36bd84c939c583
Programs	shared/types.rs shared/views.rs shared/vm_pointer.rs validation/deserialization/ast.rs validation/deserialization/mod.rs validation/deserialization/translate.rs validation/mod.rs validation/verification/ast.rs validation/verification/linkage.rs validation/verification/mod.rs validation/verification/translate.rs

Target	sui-adapter
Repository	https://github.com/MystenLabs/sui ↗
Version	eed1f9d396e67d9e70e0311fa36bd84c939c583
Programs	mod.rs spanned.rs linkage/resolved_linkage.rs linkage/mod.rs linkage/resolution.rs linkage/legacy_linkage.rs linkage/analysis.rs linkage/config.rs execution/mod.rs execution/context.rs execution/values.rs execution/interpreter.rs typing/invariant_checks/mod.rs typing/invariant_checks/defining_ids_in_types.rs typing/invariant_checks/type_check.rs typing/invariant_checks/memory_safety.rs typing/mod.rs typing/ast.rs typing/translate.rs typing/verify/mod.rs typing/verify/move_functions.rs typing/verify/input_arguments.rs typing/verify/drop_safety.rs typing/verify/memory_safety.rs loading/mod.rs loading/ast.rs loading/translate.rs env.rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 10 person-weeks. The assessment was conducted by two consultants over the course of seven calendar weeks.

Contact Information

The following project manager was associated with the engagement:

 **Chad McDonald**
Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

 **Jie Ding**
Engineer
jie@zellic.io ↗

 **Nan Wang**
Engineer
nan@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

October 16, 2025	Start of primary review period
-------------------------	--------------------------------

October 17, 2025	Kick-off call
-------------------------	---------------

December 5, 2025	End of primary review period
-------------------------	------------------------------

3. Detailed Findings

3.1. Native function memory leak in Arena-allocated Function structures

Target	jit/execution/translate.rs		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

The Function struct defined in jit/execution/ast.rs contains a native field defined as Option<NativeFunction>, where NativeFunction is Arc<UnboxedNativeFunction>. This means the UnboxedNativeFunction itself is allocated on the standard heap.

In translate.rs, the functions() method creates Function instances and allocates them into the Arena:

```
let prealloc_functions: Vec<Function> = module
    .function_defs()
    .iter()
    .enumerate()
    .map(|(ndx, fun)| {
        let findex = FunctionDefinitionIndex(ndx as TableIndex);
        alloc_function(package_context, module_name, module, findex, fun)
    })
    .collect::

```

The result loaded_functions is of type ArenaVec<Function>, which is managed by the Bump Allocator (Arena). Crucially, this means the ArenaVec is holding a pointer (the Arc) that points to the standard heap.

Given the nature of the Arena Allocator (which skips Drop implementations), any subsequent error return (such as a verification failure or a module initialization error) will lead to a memory leak of the native function. The Arc's reference count will never be decremented, and the underlying UnboxedNativeFunction on the heap will never be freed.

The codebase acknowledges this issue with a TODO comment in ast.rs:

```
// TODO(vm-rewrite): This field probably leaks.
native: Option<NativeFunction>,
```

Impact

Memory leaks can accumulate over time, eventually exhausting available memory and causing the Sui VM to crash, leading to denial of service on validator nodes.

Recommendations

Implement a custom cleanup mechanism that manually decrements Arc reference counts before Arena deallocation.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in commit [6fe4d8cf](#).

3.2. Unbounded package-cache growth can lead to memory exhaustion

Target	cache/mod.rs		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

The `MoveCache` struct contains a `package_cache` field of type `HashMap<VersionId, Arc<Package>>`. When developers deploy contracts, packages are added to this cache via the `add_to_cache()` method:

```
pub fn add_to_cache(
    &self,
    package_key: VersionId,
    verified: verification::ast::Package,
    runtime: jit::execution::ast::Package,
) {
    // NB: We grab a write lock here to ensure that we don't double-insert a
    package.
    [... ...]
    package_cache.insert(package_key, Arc::new(package));
}
```

However, there is no size limit enforced on `package_cache`, nor is there any mechanism to evict or clear entries. The `package_cache` is initialized during `MoveRuntime` initialization and persists for the lifetime of the runtime:

```
impl MoveRuntime {
    [... ...]
    let cache = Arc::new(MoveCache::new(vm_config.clone()));
    Self {
        cache,
        natives,
        vm_config,
    }
}
```

Since there is no bound on the number of packages that can be cached, and entries are never removed, this allows the `HashMap` to grow indefinitely. An attacker could exploit this by deploying a large number of packages, causing the cache to consume all available memory.

Impact

Unbounded growth of `package_cache` can lead to out-of-memory conditions, causing the Sui VM to crash and resulting in denial of service on validator nodes.

Recommendations

Implement an LRU cache eviction policy, or enforce a maximum cache size with automatic eviction when the limit is reached.

Remediation

This issue has been acknowledged by Mysten Labs.

Mysten Labs provided the following comment:

This is a known issue, and we do not intend on fixing this currently, but will keep it in mind for future improvement/fixes.

1. Introducing any type of eviction in the cache presents serious issues around determinism (esp. cross-node determinism) that would need to be handled, and issues arising from this vs the OOM possibility seem far more likely to happen.
2. We have conducted analysis of the package cache growth size, and loading every single package published on mainnet leads to only ~15-16 GB of usage (far below OOM levels for validators).
3. If for some reason the package cache were to OOM no single transaction could load a large enough number of packages to trigger the OOM in which case the killing of the process (due to OOM) would solve the OOM and the subsequent restart of the validator would allow the network to make progress.

3.3. Potential integer overflow in bytecode offset calculation

Target	jit/execution/translate.rs		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

In the `flatten_and_renumber_blocks()` function, the `current_offset` variable is calculated by accumulating the length of bytecode blocks:

```
pub(crate) fn flatten_and_renumber_blocks(
    blocks: BTreeMap<u16, Vec<input::Bytecode>>,
    jump_tables: &mut [VMPointer<VariantJumpTable>],
) -> Vec<input::Bytecode> {
    let mut offset_map = BTreeMap::new();
    let mut concatenated = Vec::new();

    let mut current_offset: u16 = 0;
    for (line_name, bytecodes) in &blocks {
        offset_map.insert(*line_name, current_offset);
        current_offset += bytecodes.len() as u16; // <--- potential overflow
        concatenated.extend_from_slice(bytecodes);
    }
    // ...
}
```

The addition `current_offset += bytecodes.len() as u16` does not use checked arithmetic. Although serialized bytecode has a limit of `0xFFFF` (65,535) on the maximum number of bytecode that can be read, optimization passes in `generate_basic_blocks()` could potentially introduce new instructions. If the total bytecode count exceeds `u16::MAX`, this would cause an integer overflow.

Impact

An integer overflow in bytecode offset calculation would cause the Move VM runtime to crash, which may eventually result in denial of service on validator nodes.

Recommendations

Replace the unchecked addition with a checked addition that raises an invariant violation error if overflow occurs.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in commit [b8784bed](#) ↗.

3.4. Assert-based panic in interpreter

Target	execution/interpreter/eval.rs		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

In the step function of the Move VM interpreter, there is an `assert!`. It is impossible to guarantee that `pc <= instructions.len()` after JIT optimization, because JIT and `code_unit` check are disconnected, which may possibly lead to panic.

```
fn step(
    state: &mut MachineState,
    run_context: &mut RunContext,
    gas_meter: &mut impl GasMeter,
) -> VMResult<StepStatus> {
    let fun_ref = state.call_stack.current_frame.function();
    let pc = state.call_stack.current_frame.pc as usize;
    let instructions = fun_ref.code();
    assert!(
        pc <= instructions.len(),
        "PC beyond instruction count for {}",
        fun_ref.name(&run_context.vtables.interner)
    );
}
```

Impact

If the `pc` exceeds `instructions.len()` after JIT, it will cause the Move VM runtime to crash, which may eventually result in a denial of service on validator nodes.

Recommendations

Modify the assertion to return `Err` instead of panicking.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in commit [6d1762e1](#) ↗.

3.5. Gas cost double-charged in native_sub_string

Target	natives/movestdlib/string.rs		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

In the `native_sub_string()` function, according to the commented calculation rules, the gas cost should be `base_cost + unit_cost * sub_string_length_in_bytes`. However, the actual implementation charges `base_cost` twice:

```

/*****
 * native fun internal_sub_string
 *
 *   gas cost: base_cost + unit_cost * sub_string_length_in_bytes
 *
 *****/
fn native_sub_string(
    gas_params: &SubStringGasParameters,
    context: &mut NativeContext,
    _ty_args: Vec<Type>,
    mut args: VecDeque<Value>,
) -> PartialVMResult<NativeResult> {
    debug_assert!(args.len() == 3);

    let j = pop_arg!(args, u64) as usize;
    let i = pop_arg!(args, u64) as usize;
    // Charge before doing work
    native_charge_gas_early_exit!(context, gas_params.base); // <--- first
    base_cost

    if j < i {
        return Ok(NativeResult::err(context.gas_used(), 1));
    }
    let cost = gas_params.base + gas_params.per_byte * NumBytes::new((j - i)
    as u64); // <--- second base_cost
    native_charge_gas_early_exit!(context, cost);
    // ...
}

```

The function first charges `gas_params.base` via `native_charge_gas_early_exit!` and then calculates `cost`, which includes `gas_params.base` again. This results in an actual gas cost of $2 * \text{base_cost} + \text{unit_cost} * \text{sub_string_length_in_bytes}$, which does not match the documented formula.

By comparison, `native_index_of()` correctly charges `base_cost` only once at the beginning, then charges additional per-byte costs separately.

Impact

Users are overcharged for `sub_string` operations. While this does not pose a security risk, it represents inconsistency between documentation and implementation.

Recommendations

Either update the gas calculation to match the documented formula, or update the comment to reflect the actual behavior.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in commit [bd8474d1](#).

4. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

4.1. Overview

This project implements a static programmable transactions execution stack, employing a layered pipeline architecture for the full life-cycle management of Move programs: load → syntax abstraction → type checking → semantic verification → execution-time binding → interpretive execution. This system ensures that smart contract code undergoes strict multilayer verification and optimization before on-chain execution, guaranteeing both execution security and near-native code performance.

The entire system is driven by seven core commands in the SPT, enabling a complete smart contract execution flow, including Move package deployment, Move package upgrade, Move function calls, Coin merging, object transfer, Coin splitting, and dynamic container creation.

In a normal operating state, commands are executed sequentially, modules are loaded and verified in order, and the protocol progresses efficiently. In exceptional circumstances (such as missing dependencies, type mismatches, or insufficient balance), the system uses multilayer error checking and rollback mechanisms to quickly identify the issue and abort execution, thereby ensuring system security.

SPT preexecution processing: From external transaction to internal AST

This involves the following two phases.

Preexecution processing (P → L AST abstraction)

This phase transforms the external `ProgrammableTransaction` structure (P::) into the internal abstract syntax tree (L::). The core tasks are type sanitization and dependency resolution. The system loads all object inputs, parses their runtime ownership and Move types, and for `Publish` and `Upgrade` commands, executes complex linkage analysis to compute precise external address mappings, providing complete type and dependency information for subsequent phases.

Compilation-time verification (L → T AST refinement and validation)

This is the security core of the execution stack. It transforms the L::AST into the final executable T::AST through three steps: translation, verification, and invariant checks.

1. **Translation.** It performs initial type inference and executes scope reference analysis (e.g., marking Move operations).
2. **Verification.** It executes strict semantic verification (input parameters, function signatures), memory safety (borrowing rules), and resource safety (drop_safety) checks.
3. **Invariant checks.** This involves final type checking and core invariant validation, ensuring that the code's runtime behavior on the Move VM is entirely predictable and safe.

Seven core commands

The execution stack primarily revolves around seven command types: Publish, Upgrade, MoveCall, TransferObjects, SplitCoins, MergeCoins, and MakeMoveVec. Each command type has its specific purpose and trigger conditions. Below, we detail their functions, execution timing, verification logic, and impact on the state machine upon successful verification.

4.2. Command type: Publish

This section outlines the structure, basic function, and life cycle of the Publish message.

Message structure and basic function

The Publish command carries a module deployment request from a developer and marks the starting point for smart contracts on chain. The Publish command typically includes the compiled module bytecode, a list of dependency package identifiers (dep_ids), external dependency mappings (linkage), and the deployer's signature. Its core function is to securely deploy locally compiled Move code onto the chain and initiate the module initialization process. The Publish command ensures legal linking with previously verified modules or handles missing dependencies. If all dependencies are deployed, the new module references them to build the complete package structure; if some dependencies are missing, the deployment process is rejected to ensure system consistency.

Message life cycle

The message undergoes the following steps.

Creation and propagation

At the start of deployment, the client packages the locally compiled module bytecode to construct the Publish command. The trigger condition is when a developer initiates a deployment request via a CLI or SDK, prompting the transaction builder to create and send the Publish command. After

creation, the client submits it to the blockchain network. Normally, each deployment has a unique initiator sending `Publish`, and validating nodes receive `Publish` to participate in subsequent verification and state updates. When the network is healthy and the initiator is honest, most deployments begin with a normal `Publish` submission.

Reception and verification

When a validating node receives the `Publish` command, it verifies its legitimacy via the signature module, including the following verification steps:

1. **Message integrity verification.** It uses the transaction-signing mechanism to verify that the `Publish` command's digital signature was issued by the declared initiator, ensuring message integrity and credibility. Through the account system, it checks if the initiator has sufficient gas and necessary permissions to execute the deployment.
2. **Module deserialization and bounds check.** The client's compiled modules are deserialized into `CompiledModule` structures according to version specifications. Strict bounds checking is performed, verifying that all index references do not exceed the corresponding table limits and checking that the bytecode length matches the declared function body size, defensively detecting potential buffer-overflow attacks. This is the first line of defense against malicious bytecode, preventing attack code attempting to exploit parser vulnerabilities early on.
3. **Dependency handling and linkage construction.** A unique `original_id` is generated for the new package, and all direct and transitive dependencies are recursively loaded from the `package_store` based on the `dep_ids` list. The existence and version compatibility of the dependency packages are verified. The current module is combined with the dependencies into a unified `MovePackage` structure. Since the `package_id` cannot be determined before publication (it depends on the transaction hash), the symbolic references provided by the user during compilation (module name and function name) need to be resolved to actual package addresses and offsets, using placeholders that are replaced in a later stage. This deferred binding mechanism allows packages to be reused in different environments, only requiring dependency resolution at deployment time.
4. **Dependency validation.** Deep semantic verification is performed on the constructed dependencies. Linkage integrity is validated, ensuring that all external references find corresponding definitions, type signatures are consistent across dependency boundaries, and generic constraints are satisfied. A topological sorting algorithm is used to detect loops in the dependency graph, preventing circular dependencies like $A \rightarrow B \rightarrow C \rightarrow A$, which could cause initialization deadlocks and stack overflows. It checks that dependency package versions meet the requirements of the current package and validates that interface changes are backward compatible. Although dependencies are loaded in the dependency-handling phase, global consistency checks only occur once all dependencies are ready.
5. **JIT optimization and memory reorganization.** This is the most complex optimization phase, aimed at improving execution efficiency by reorganizing memory layout. A

contiguous Arena memory region is allocated within each Package object, and all module metadata (handles, instantiations) are migrated to this region to improve cache locality, simplify memory management (unified freeing), and reduce memory fragmentation. Function optimization is key, including function table flattening (collecting dispersed functions into a unified table), V-Table construction (assigning a global index to each function to support $O(1)$ lookups), in-line candidate identification (marking simple functions as in-line candidates), and bytecode relocation (converting relative jump addresses to absolute addresses within the Arena). Through the V-Table, function calls are optimized from $O(n)$ linear search to $O(1)$ hash table lookup, significantly reducing cross-module call overhead.

6. **Bytecode final check.** Despite multiple rounds of prior verification, bytecode-level checks remain critical. Execution-stack balance analysis simulates the execution of each instruction to track the operand stack depth, ensuring the stack is exactly empty (or contains only return values) at the function end, checking for potential stack overflows or underflows. Control flow integrity is verified, ensuring that all jump targets are valid and do not jump outside the function and checking the completeness of exception-handling paths. It verifies that Move resources are not illegally copied or dropped and checks that reference lifetimes do not violate borrowing rules. It scans for instruction combinations that could trigger undefined behavior, preventing the use of deprecated or unsafe instructions.

Only upon passing these verifications is the Publish command considered valid.

State update and subsequent operations

When a node accepts a valid Publish command, it performs the following operations:

1. **Persistence to storage.** The optimized Package object is serialized and written to the `package_store`, updating package indexes for quick lookup and recording package metadata (deployment time, dependency versions, etc.). Once published, the package is accessible to all nodes in the network; other packages can immediately declare dependencies on it, and users can call its public functions. The content of a published package is immutable; upgrades require a separate Upgrade command, ensuring the stability of dependent package behavior.
2. **Module initialization.** Move allows each module to define a special `init` function, which is automatically executed upon the module's first deployment. The VM calls the `init` function of each module sequentially according to their dependency order, ensuring that dependencies are initialized before the current module. Execution occurs in a sandboxed environment with restricted gas usage, and `init` functions are commonly used for creating global singleton objects, registering capabilities, transferring administrative authority objects to the deployer, and so on. The automatic invocation of the `init` function simplifies the module deployment process, preventing state inconsistency caused by users forgetting to initialize.

Thus, the Publish command triggers the validating node's package storage update and module-initialization execution. If verification fails for any reason (signature error, missing dependency, bounds check failure, circular dependency, bytecode violation, etc.), the node rejects the Publish command and does not execute initialization, thereby ensuring protocol security.

4.3. Command type: MoveCall

This section outlines the structure, basic function, and life cycle of the MoveCall message.

Message structure and basic function

The MoveCall command represents a request to call an already deployed Move function, carrying call parameters and the function signature. Its core function is to allow the execution engine to execute business logic on verified code, thereby providing the actual functionality of smart contracts. The call includes target package ID, module name, function name, type arguments, and function arguments, used for subsequent function lookup and execution. The MoveCall command typically does not require network-wide verification like Publish but is handled within the execution environment of a single node. This allows the execution engine to perform quick function lookups via the V-Table, avoiding the overhead of broadcasting every call and reducing message complexity.

Message life cycle

The message undergoes the following steps.

Creation and propagation

The trigger condition is when a user initiates a function call via a transaction, and the call parameters are successfully prepared within the SPT context. Normally, a single transaction may contain multiple MoveCall commands, each triggering an independent function execution. The execution node parses the MoveCall command when processing the transaction; if conditions are met, it uses the execution engine to prepare the execution environment and generate the function call. The user submits the transaction containing the MoveCall to the validating node for execution.

Reception and verification

When the execution engine receives the MoveCall command, verification includes the following steps:

1. **Parameter processing.** It translates the function arguments provided by the user into CtxValue according to their abilities and location. It verifies the type matching between

arguments and the function signature, ensuring that the number of arguments, types, and ownership semantics comply with the function declaration.

2. **Environment construction.** The function call uses the information provided by the user (package ID, module name, function name) obtained during input processing to fetch the relevant packages. It loads necessary packages from the `package_store` and retrieves their V-Tables to build the Move VM execution environment. It ensures all dependent packages are loaded and accessible.
3. **Function lookup.** Based on the function name, package ID, module name, and so on, the final function to be called is retrieved from the V-Table. The function handle is quickly located via an $O(1)$ hash table lookup, fetching metadata such as the bytecode location in the Arena, function signature, number of generic parameters, and local variable slots.
4. **Type instantiation.** If the function is generic, type instantiation is performed based on the type arguments provided in the call, verifying that the type parameters satisfy the function's generic constraints (e.g., copy, drop, store, key abilities).

State update and subsequent operations

Upon retrieving a valid function, the execution engine executes the function call. The call process is handled separately depending on whether the function is a native function:

- **Native function execution.** If it is a native function (e.g., `std::vector::push_back`, `sui::transfer::transfer`), the built-in Rust implementation is called directly. Native functions provide high-performance basic operations, avoiding bytecode interpretation overhead.
- **Bytecode interpretive execution.** If it is not a native function, the function's instructions are retrieved and executed step-by-step in the interpreter. The interpreter maintains an operand stack and local variable array. Precise gas metering (`charge gas`) is performed for every instruction, accumulating the execution cost to prevent infinite loops or resource-exhaustion attacks. Execution instructions include but are not limited to arithmetic operations (`Add`, `Sub`, `Mul`), stack operations (`Pop`, `CopyLoc`, `MoveLoc`), control flow (`Branch`, `BrTrue`, `BrFalse`), function calls (`Call`, `CallGeneric`), and object operations (`MoveFrom`, `MoveTo`, `BorrowGlobal`).

After execution, the value from the operand stack is returned as the function call's return value. The return value is converted back to `CtxValue` form, which can be used in subsequent SPT commands. The entire execution process occurs in a sandboxed environment, ensuring memory safety, type safety, and resource safety.

If an error occurs during execution (e.g., stack overflow, gas exhaustion, resource conflict, type error), the execution engine aborts execution and rolls back state changes, returning an execution error. This ensures that even if a function execution fails, the blockchain state remains consistent.

4.4. Command type: TransferObjects

This section outlines the structure, basic function, and life cycle of the TransferObjects command.

Message structure and basic function

The TransferObjects command is used for the batch-transfer mechanism of object ownership. When the execution engine needs to transfer the ownership of multiple objects simultaneously to a specified recipient, the TransferObjects command is sent to coordinate that "ownership of this batch of objects has changed", thereby triggering a state update. These commands carry a list of object references, the recipient's address, and type information, providing the necessary data for subsequent state tree updates.

Furthermore, TransferObjects ensures the atomicity of ownership transfer: either all objects are successfully transferred, or all fail and roll back. This prevents state inconsistency caused by partial transfers.

Message life cycle

The message undergoes the following steps.

Creation and propagation

The trigger condition is when an ownership-transfer operation needs to be executed within the SPT context. Specifically, when `sui::transfer::transfer` or a similar ownership-transfer function is executed, the SPT parser generates the TransferObjects command. The command includes a list of objects to be transferred (each containing value and type) and the recipient's address, and it is processed within the execution context without network broadcast.

Reception and verification

When the execution engine processes the TransferObjects command, verification includes the following steps:

1. **Parameter extraction.** The objects parameter is iterated, and the type information of each object is collected to construct the `object_tys` vector. Object references are parsed in batch via `context.arguments()` to retrieve `object_values`. The recipient parameter is parsed into an `AccountAddress`.
2. **Consistency check.** The `assert_invariant!` is used to ensure the number of objects strictly matches the number of types, preventing type confusion. It verifies that all objects possess the key ability (only objects with the key ability can be stored and transferred).

3. **Ownership verification.** It checks whether the current execution context owns these objects, ensuring that objects not belonging to the sender are not transferred.

State update and subsequent operations

Upon successful verification, the execution engine performs the transfer operation on each object:

1. **Ownership wrapping.** The recipient's address is wrapped into the `Owner::AddressOwner` enum type.
2. **State tree update.** Then, `context.transfer_object()` is called to modify the object's ownership field. The object's belonging relationship is updated in the state tree (object runtime), removed from the sender's object set, and added to the recipient's object set.
3. **Event recording.** An ownership-transfer event is generated, recording the transferred object ID, sender, recipient, and so on, facilitating state change tracking by off-chain applications.

The command returns an empty vector because the transfer operation does not produce new output values; its effect is manifested in the state change. Moreover, `TransferObjects` strictly adheres to Sui's ownership system; each object can have only one owner at any time, and ownership transfer is atomic, preventing double-spend attacks.

4.5. Command type: `SplitCoins`

This section outlines the structure, basic function, and life cycle of the `SplitCoins` command.

Message structure and basic function

The `SplitCoins` command is used for splitting `Coin` objects, representing the division of a single `Coin`'s balance into multiple independent `Coins`. Its core function is to allow the execution engine to safely split `Coins` when change or fund allocation is needed, supporting flexible asset operations. These commands carry the source `Coin` reference, a list of split amounts, and type information, used for subsequent balance calculation and new `Coin` creation.

Message life cycle

The message undergoes the following steps.

Creation and propagation

The trigger condition is when multiple smaller `Coins` need to be split from a single `Coin` within the SPT context. Common scenarios include payment (splitting a precise payment amount from a

large Coin), airdrop (splitting multiple rewards from a reward pool), and change (retaining change after splitting). The command includes the source Coin reference and an array of split amounts, and it is processed within the execution context.

Reception and verification

When the execution engine processes the `SplitCoins` command, verification includes the following steps:

1. **Execution tracking (optional).** If `TRACK_EXECUTION` mode is enabled, the source Coin is added to the list of objects to be updated, used for subsequent state synchronization and dry-run analysis.
2. **Parameter parsing.** The mutable reference to the Coin to be split is obtained via `context.argument(coin)`. And `amounts` is parsed as `Vec<u64>`, where each element represents the amount of a new Coin.
3. **Overflow check.** All split amounts are accumulated, and `checked_add` is used to prevent integer overflow. If `amount_1 + amount_2 + ... + amount_N` exceeds `u64::MAX`, a `CoinBalanceOverflow` error is returned. This prevents attackers from forging enormous Coins through overflow.
4. **Balance validation.** The current balance of the source Coin is obtained via `coin_ref_value()`. The `fp_ensure!` macro is used to verify that the source balance is greater than or equal to the total split amount. If the balance is insufficient, an `InsufficientCoinBalance` error is returned, including detailed balance and requirement information.

State update and subsequent operations

Upon successful verification, the execution engine performs the split operation:

1. **Balance deduction.** Here, `coin_ref_subtract_balance(total)` is called to deduct the total split amount from the source Coin. This is an atomic operation; either the entire deduction succeeds, or it fails and rolls back.
2. **Creation of new Coins.** Here, `context.new_coin(amount)` is called for each amount in `amounts` to create independent Coin objects. Each new Coin has a unique object ID, and its initial owner is the current execution context. The new Coins have the same type as the source Coin (e.g., `Coin<SUI>`).
3. **Return new Coins.** A vector of the newly created Coin objects is returned. These Coins can be used in subsequent SPT commands (e.g., transferred to other users, merged into other Coins, or passed as function arguments).

The entire process ensures balance conservation — source Coin deduction equals total new Coins amount. The split operation is atomic, preventing the scenario where the balance is successfully deducted but the new Coin creation fails.

4.6. Command type: MergeCoins

This section outlines the structure, basic function, and life cycle of the MergeCoins command.

Message structure and basic function

The MergeCoins command is used for merging Coin objects, representing the aggregation of multiple Coin balances into a single target Coin. Its core function is to allow the execution engine to consolidate dispersed funds, simplifying asset management. These commands carry the target Coin reference, a list of source Coins, and balance information, used for subsequent balance accumulation and object destruction.

Message life cycle

The message undergoes the following steps.

Creation and propagation

The trigger condition is when multiple Coins need to be merged into one within the SPT context. Common scenarios include fund consolidation (merging multiple small Coins into a large one), gas fee optimization (reducing the number of held objects to lower storage costs), and cleanup operations (consolidating remaining Coins after a batch operation). The command includes the target Coin reference and an array of source Coins and is processed within the execution context.

Reception and verification

When the execution engine processes the MergeCoins command, verification includes the following steps:

1. **Execution tracking (optional).** If tracking mode is enabled, the target Coin is added to the list of objects to be updated.
2. **Parameter parsing.** The target is parsed as the mutable reference to the target Coin. And coins are parsed in batch as the list of Coin objects to be merged. It verifies that all Coins have the same type (e.g., all `Coin<SUI>`); merging Coins of different types is disallowed.
3. **Destruction and extraction.** Each source Coin undergoes a destruction operation — `context.destroy_coin()` is called to destroy the Coin object, and its balance value is

extracted and collected into the `amounts` vector. The destroyed Coin is removed from the state tree, and its object ID is marked as deleted. The destruction operation ensures source Coins cannot be double-spent.

4. **Overflow check (accumulation phase).** All source Coin balances are accumulated ($\text{additional} = \text{amount_1} + \text{amount_2} + \dots + \text{amount_N}$), and `checked_add` is used to detect overflow.
5. **Overflow check (final balance).** The current balance of the target Coin, `target_value`, is obtained, and it verifies that $\text{target_value} + \text{additional} \leq \text{u64::MAX}$, preventing the final balance from overflowing.

State update and subsequent operations

Upon successful verification, the execution engine performs the merge operation:

1. **Balance addition.** Here, `coin_ref_add_balance(additional)` is called to add the accumulated value to the target Coin. This is an atomic operation, ensuring the balance update succeeds.
2. **State cleanup.** Source Coins have already been destroyed during the verification phase, requiring no extra cleanup. The new balance of the target Coin is reflected in the state tree.
3. **Return value.** An empty vector is returned because source Coins have been destroyed, and the result of the operation is reflected in the increased balance of the target Coin.

The entire process ensures balance conservation; the target Coin increase equals the total source Coins' destroyed amount. Merging prevents double-spending; source Coins cannot be used again after destruction. The operation is atomic, preventing the scenario where source Coins are destroyed but the target Coin balance is not increased.

4.7. Command type: MakeMoveVec

This section outlines the structure, basic function, and life cycle of the `MakeMoveVec` command.

Message structure and basic function

The `MakeMoveVec` command is used for the runtime dynamic creation of the `Move` vector type. Its core function is to allow the execution engine to construct dynamic arrays without relying on compile-time constants, supporting generic element types. These commands carry the element type identifier, a list of element values, and capacity information, used for subsequent type-safe packaging and memory allocation.

Message life cycle

The message undergoes the following steps.

Creation and propagation

The trigger condition is when a dynamic vector needs to be constructed as a function parameter or data structure within the SPT context. Common scenarios include batch parameter passing (constructing `vector<address>` as the recipient list for a batch transfer), dynamic configuration (constructing `vector<u64>` as configuration parameters), and nested structures (constructing `vector<vector<u8>>` for complex data). The command includes the element type and an array of element values, and it is processed within the execution context.

Reception and verification

When the execution engine processes the `MakeMoveVec` command, verification includes the following steps:

1. **Parameter parsing.** The `ty` parameter specifies the vector's element type, which can be a primitive type (`u64`, `address`, `bool`), a custom struct, or a nested vector. All element values are parsed into a list of `CtxValue` via `context.arguments(items)`.
2. **Type validation.** It verifies that the runtime type of all elements is strictly consistent with the declared `ty`. If declared as `vector<u64>`, all elements must be of type `u64`; mixing in other types is disallowed. For custom structs, the full type signature (package name, module name, struct name, generic arguments) is verified.
3. **Ability check.** It verifies that the element type possesses the necessary abilities. Vector element types must have either the copy or drop ability, ensuring the vector can be safely copied or discarded.

State update and subsequent operations

Upon successful verification, the execution engine creates the vector:

1. **Memory layout.** Here, `CtxValue::vec_pack(ty, items)` is called to organize the data according to Move's vector memory model. The vector is represented in memory as length field (`u64`), capacity field (`u64`), and data pointer (pointing to the element array).
2. **Capacity allocation.** Appropriate capacity is preallocated based on the number of elements, usually slightly greater than the current number of elements to reserve space for potential subsequent push operations.

A vector containing a single element is returned, which is the newly created `vector<T>` object. This vector can be used in subsequent SPT commands, such as being passed as an argument to `MoveCall`.

The created vector is type-safe, guaranteeing no type confusion at runtime. Vectors support dynamic growth (via `vector::push_back`) and shrinking (via `vector::pop_back`), with memory management handled automatically by the VM.

4.8. Command type: Upgrade

This section outlines the structure, basic function, and life cycle of the Upgrade command.

Message structure and basic function

The Upgrade command is used for the secure, authorized, and atomic content update of an already deployed Move package. It is not a standard `Publish` but a special deployment requiring proof of authority. Its core function is to enable code iteration and maintenance while strictly adhering to Move's backward-compatibility rules.

The command carries the new version's module bytecode (`module_bytes`), a list of dependency IDs (`dep_ids`), the current package ID (`current_package_id`), external dependency mappings (`linkage`), and the crucial upgrade ticket (`upgrade_ticket`). The upgrade ticket is the credential that locks permission and content, ensuring only authorized and untampered updates can be executed.

Message life cycle

The message undergoes the following steps.

Creation and propagation

The trigger condition is when a package administrator, having obtained the `upgrade_ticket` through a dedicated Upgrade transaction, initiates an upgrade request containing the new code. This command is typically triggered only when publishing a backward-compatible update, fixing a vulnerability, or adding new features. The upgrade operation is processed within the execution context and relies on the execution engine's permission-verification capabilities.

Reception and verification

When the execution engine receives the Upgrade command, it executes a multistage verification gate to ensure the legality and safety of the upgrade:

1. **Ticket acquisition and permission check.** It retrieves and validates the `upgrade_ticket` from the transaction context.
2. **Strict package-ID matching.** It strictly requires the package ID provided by the client to match the package ID locked in the ticket, preventing misuse or malicious upgrading of other packages (target consistency check).
3. **Module deserialization.** It deserializes the new version's bytecode, prepares the execution environment, and accounts for gas consumption at this step.
4. **Content digest validation.** It computes the digest of the new modules and dependencies and strictly compares it to the digest prestored in the ticket. It ensures the submitted bytecode content has not been tampered with and is identical to the authorized content (content integrity check).

Only upon passing all the above strict verifications is the upgrade request considered safe and valid.

State update and subsequent operations

Once all prerequisite verifications pass, the execution engine performs the core package upgrade operation:

1. **Execute the atomic upgrade.** Here, `context.upgrade()` is called. This function is responsible for writing the new modules to the `package_store`, replacing the old version's content. The upgrade operation strictly adheres to the compatibility policy defined in `upgrade_ticket.policy` (e.g., whether field deletion or type signature changes are allowed).
2. **Generate the upgrade receipt.** Upon a successful upgrade, the system generates and returns an upgrade receipt (`upgrade_receipt`). The receipt serves as proof of a successful transaction, containing the consumed `upgrade_ticket` and the upgraded package ID (which remains unchanged during the upgrade). The command ultimately returns a vector containing the receipt, which can be used for subsequent operations or off-chain tracking.

4.9. Attack surface

This section outlines the attack surface.

Client-side bytecode tampering risk

There is no mechanism to guarantee that the compiled bytecode has not been modified after being generated by the SUI client on the user's side before deployment.

An attacker could exploit this by modifying the compiled bytes to introduce malicious code or exploit parsing vulnerabilities in the system.

Memory leak due to cross-memory referencing

The Arena memory area manually manages its own memory and explicitly bypasses Rust's automatic garbage collection. If data structures within the Arena contain pointers or references to memory allocated on the standard Rust heap, the heap-allocated memory will not be automatically freed upon the Arena's disposal.

This results in memory leaks of the referenced heap memory.

JIT cache exhaustion

The JIT cache is implemented as a HashMap storing optimized package objects, but as of the time of writing, it lacks any implementation for removal or eviction of entries.

The cache will grow indefinitely as new packages are introduced, eventually leading to an out-of-memory error, causing a persistent denial-of-service condition.

Use-after-free risk during cache eviction

Since the Arena's memory is manually managed, if a future fix implements package removal from the JIT cache, the manual deallocation of the package's Arena memory could occur while other parts of the VM still hold cloned or referenced pointers to the now-freed Arena region.

This creates a significant risk of use-after-free vulnerabilities if the cache-removal logic is flawed.

Execution panic due to PC bound check failure

In the interpreter's execution loop, the `assert!` check for the program counter boundary is unsafe. If PC is greater than `instructions.len()`, it triggers a direct Rust panic!.

A well-crafted transaction could intentionally cause this panic, leading to a system crash.

Bytecode overflow risk during JIT flattening

During the JIT optimization phase, specifically in `flatten_and_renumber_blocks`, the bytecodes from different blocks are concatenated in a loop.

The process does not ensure that the sum of the bytecodes' lengths will not exceed the maximum allowable size for the resulting function body, potentially leading to integer overflow or unexpected behavior.

Native function argument injection

The arguments passed to Native functions are directly controlled by the user via the transaction input.

If the internal logic or input validation within a specific Native function is insufficiently rigorous, it could be exploited by an attacker to trigger logic flaws or security issues not covered by Move's type system.

JIT causes the original safety assumptions to fail

The JIT opt section may completely alter the original bytecode structure, which could cause the optimized code to invalidate some original safety assumptions.

If the bytecode's structure and order are altered after JIT processing, some existing security checks will become invalid, potentially leading to unexpected behavior or even a panic.

5. Assessment Results

During our assessment on the scoped VM Rewrite targets, we discovered five findings. No critical issues were found. Four findings were of high impact and the remaining finding was informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.